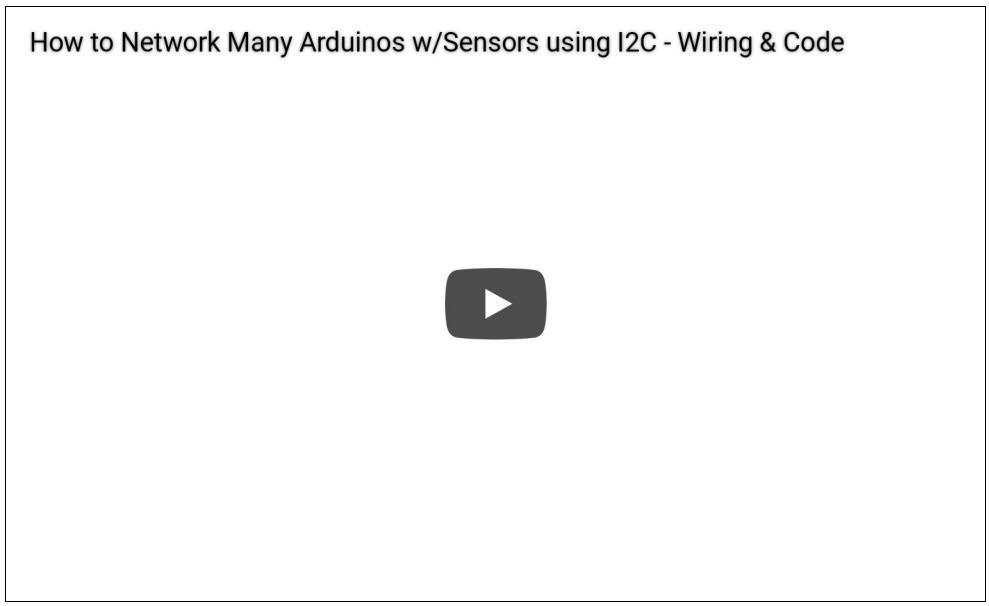
TechBitar (/)

About (/)
Projects (/projects.html)
Contact (/contact.html)
Blog (/blog.html)

How to Network Many Arduinos w/Sensors using I2C



UPDATES:

- March 29, 2014: I have improved and compacted the code

In this example, I have wired five Arduino (ATmega328) microcontrollers together via I2C protocol (http://arduino.cc/en/reference/wire). Four Arduino slave units read from accompanying sensors then send their respective sensors' data to a master Arduino unit for processing. The Arduino example code has been enhanced with arrays and min/max constants so now it can be used with few modifications to support any number of Arduino I2C nodes with any size payload, within the limitations of I2C in a Master Reader/Salve Sender configuration. When it comes to I2C, the headache is not in the wiring which is simple, but in synchronizing the I2C nodes and data transmission. I hope my code can simplify some of your Arduino networking.

BACKGROUND

If you want to connect many sensors and other components to your microcontroller but you don't have enough pins, or enough memory to run the needed libraries and code, or the microcontroller is just not fast enough to juggle the computations, you can always swap the old microcontroller with a more powerful one.

Or, depending on your project design, you simply modularize your circuit by spreading your sensors and components over multiple microcontrollers. Then network the microcontrollers using a protocol such as I2C.

Recently, I had to build a prototype made up of many components and sensors. The wiring and coding to glue all of these parts together were cumbersome. To make matters more frustrating my Arduino ran out of memory. So I decided to divide my circuit the way I divide large programs into smaller subroutines.

Instead of one Arduino with a dozen or so sensors and components attached to it, now I have five Arduinos each supporting one or two sensors. The sensor data is then sent to the master Arduino unit to do integration calculations and I/O.

By pairing key components with a microcontroller and programming it to send data via I2C to a central microcontroller, I have modularized my design making it simpler to construct and debug.

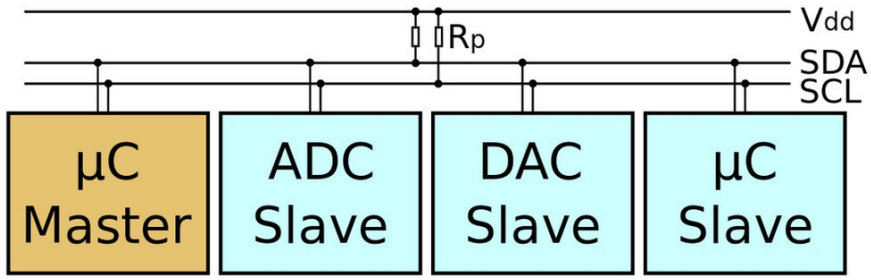
This is how many gadgets are built today, from cell phones to computers. Components from various vendors plug into a network such as I2C instead of being wired into pins of central microcontrollers.

Most of my code now is decentralized. Each of my I2C networked component has enough processing power dedicated to it. When it's ready, it sends its data to my central Arduino using only 2 wires, one for data and the other for timing.

Better yet, these components can be hot-swapped. I2C specifications make it possible to plug and unplug I2C nodes into the bus while its running. Cool! I tested this feature by unplugging then plugging slave nodes, and the network kept working. This is not exhaustive testing of the hot-swap feature of I2C, but it was interesting to try.

The price of microcontrollers today is so low you can get an Atmel microcontroller with I2C support for a couple of dollars. That's peanuts when compared to the savings in time (and money) as will as the fault tolerance you gain by modulrazing your system design instead of a monolithic microcontroller with a dozen components attached and managed by complex program with a single point of failure.

What's I2C?

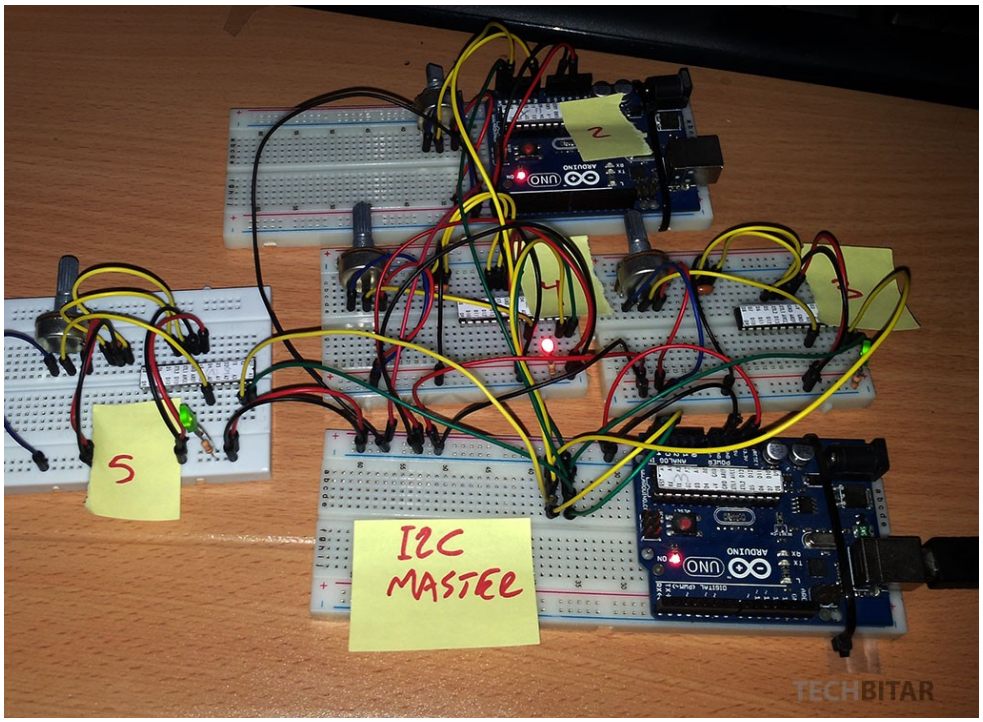


(/uploads/2/0/3/1/20316977/5037279_orig.jpg)

Philips Semiconductors (now NXP Semiconductors) developed a simple bidirectional 2-wire bus for efficient inter-IC control. This bus is called the Inter-IC or I2C-bus. Only two bus lines are required: a serial data line (SDA) and a serial clock line (SCL). Serial, 8-bit oriented, bidirectional data transfers can be made at up to 100 kbit/s in the Standard-mode, up to 400 kbit/s in the Fast-mode, up to 1 Mbit/s in the Fast-mode Plus (Fm+), or up to 3.4 Mbit/s in the High-speed mode. The Ultra Fast-mode is a uni-directional mode with data transfers of up to 5 Mbit/s

I2C for Arduino supports 127 attached devices via pins Analog 04 (SDA) and Analog 5 (SCL). The I2C pins may vary on different Arduino boards.

This is not a tutorial of I2C. If you are interested in learning more see the resources section below and visit this comprehensive I2C tutorial by John Boxall
http://tronixstuff.com/2010/10/20/tutorial-arduino-and-the-i2c-bus/ (http://tronixstuff.com/2010/10/20/tutorial-arduino-and-the-i2c-bus/)



(/uploads/2/0/3/1/20316977/4108067_orig.jpg)
This is the I2C bus connecting 5 Arduino Uno circuits, 2 full Arduinos and 3 "barebone" Arduinos.

Components

This is the list of components I have used in this project. Your project may have different requirements.

- ATmega328 microcontroller X 3 : I bought these from ebay then I burned the Optiboot Arduino bootloader on each. You can use any microcontroller with I2C. You can also use the miniature Arduinos that are becoming popular such as the Nano.
- Arduino Uno X 2: I used one Arduino Uno as I2C Master and the other as I2C Slave. Once the master/slave code was tested, I then used the Slave Arduino to program the plain ATmega328 microcontroller. It's just a convenient workflow for me.
- Potentiometer (10K Ohms) X 4 : These pots act as sensors place holders. Once the I2C network is debugged an operational, I can swap the pots with other components like temperature sensor or sound sensor. The 10K value is really not critical. You can try other values.
- Ceramic Resonator 16Mhz X 3: While these resonators are not as accurate as the 16Mhz crystals, the worked fine for my setup. They also save on parts since the 16Mhz crystal needs two capacitors but this resonator works without any support components.
- LED X 3: It's nice to have these LEDs to know when wire is applied to the microcontroller.
- Resistor 1K Ohm X 3: Used in series with the LEDs.
- Resistor 10K Ohm X 2: Pull-up resistors for the I2C SDA SCL lines. See my notes on pull-up resistors in the schematics section.
- Breadboards & Jumper wires

The Schematics

In this example, I am using 5 Arduinos. 1 acting as a master unit (to use I2C lingo) and 4 as slave units. The slave Arduinos wait for the master Arduino to request data then send it promptly.

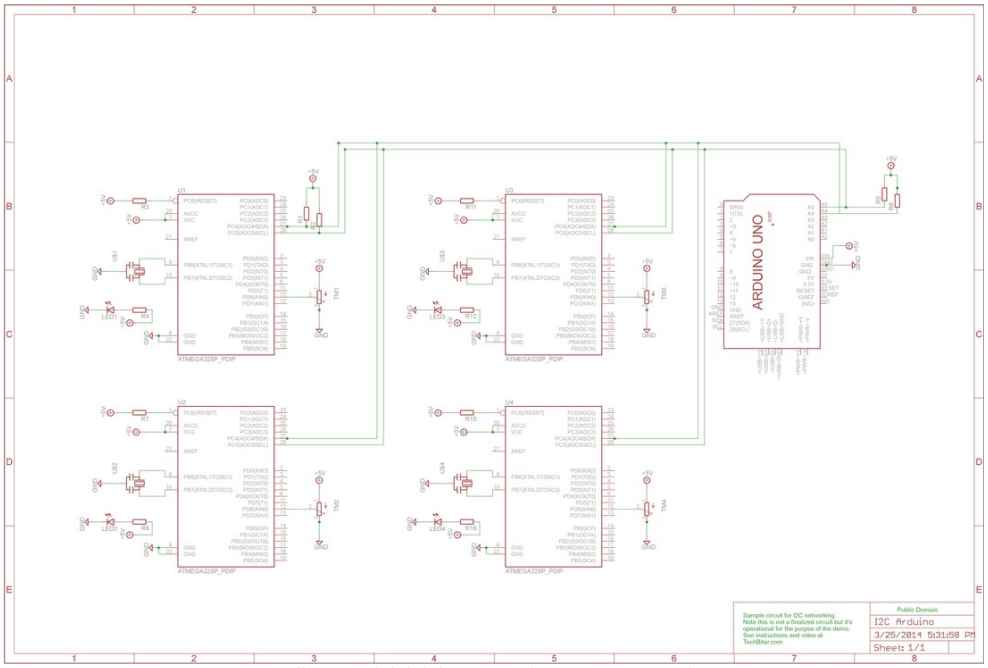
I am using a potentiometer plugged into the Analog 0 pin of each slave Arduino to simulate a sensor.

By turning the pot which is wired as a voltage divider, I will change the value of Analog 0. The code in each slave node will read Analog 0 and send its value to the master Arduino.

PULL-UP RESISTORS

The I2C specifications call for two pull-up resistors one for the SDA line and the second for the SCL line. For this demo circuit, initially I did not use any resistors just to test the stability. Then I used 10K Ohm pull-up resistors. I did not run into any issues in the short time I have tested the circuit. But in a production environment, the two pull-up resistors values must be calculated according to a formula explained in section (http://www.nxp.com/documents/user_manual/UM10204.pdf)7.1 Pull-up resistor sizing (http://www.nxp.com/documents/user_manual/UM10204.pdf) (pdf) of the official I2C manual by NXP.

For hi-res schematic, download attached file i2c_arduino.png



(/uploads/2/0/3/1/20316977/8808728_orig.png)
This is the I2C bus connecting 5 Arduinos. It's a prototype circuit and not a production circuit.

(/uploads/2/0/3/1/20316977/i2c_arduino.png)
i2c_arduino.png
Download File (/uploads/2/0/3/1/20316977/i2c_arduino.png)

The Firmware

I have two programs: one for the master Arduino, and a second for all the slave Arduinos. Before uploading the program to each slave Arduino, change the node I2C address number to make sure each Arduino node on the I2C bus can be identified with a unique address. The addresses are in HEX.

The example code is ready to upload to any number of I2C nodes (within the 127 node limit). Simply change the #define constants to change the maximum nodes and the node payload size.

Arduino has a bundled library for I2C communications named Wire.h . The related examples are simple enough to understand and modify.

Program: I2C Master Reader

```
// Program: I2C master reader template for multi-node Arduino I2C network
// Programmer: Hazim Bitar (techbitar.com)
// Date: March 30, 2014
// This example code is in the public domain.

#include <Wire.h>

#define PAYLOAD_SIZE 2 // how many bytes to expect from each I2C salve node
#define NODE_MAX 6 // maximum number of slave nodes (I2C addresses) to probe
#define START_NODE 2 // The starting I2C address of slave nodes
#define NODE_READ_DELAY 1000 // Some delay between I2C node reads

int nodePayload[PAYLOAD_SIZE];

void setup()
{
  Serial.begin(9600);
  Serial.println("MASTER READER NODE");
  Serial.print("Maximum Slave Nodes: ");
  Serial.println(NODE_MAX);
  Serial.print("Payload size: ");
  Serial.println(PAYLOAD_SIZE);
  Serial.println("**********************");

  Wire.begin();        // Activate I2C link
}

void loop()
{
  for (int nodeAddress = START_NODE; nodeAddress <= NODE_MAX; nodeAddress++) { //
    Wire.requestFrom(nodeAddress, PAYLOAD_SIZE);    // request data from node#
    if(Wire.available() == PAYLOAD_SIZE) {  // if data size is avaliable from nod
      for (int i = 0; i < PAYLOAD_SIZE; i++) nodePayload[i] = Wire.read();  // ge
      for (int j = 0; j < PAYLOAD_SIZE; j++) Serial.println(nodePayload[j]);   //
      Serial.println("************************");
      }
    }
    delay(NODE_READ_DELAY);
}
```

Program: I2C Slave Sender

```
// Program: I2C slave sender template for multi-node Arduino I2C network
// Programmer: Hazim Bitar (techbitar.com)
// Date: March 30, 2014
// This example code is in the public domain.

#include <Wire.h>

#define NODE_ADDRESS 2  // Change this unique address for each I2C slave node
#define PAYLOAD_SIZE 2 // Number of bytes  expected to be received by the master

byte nodePayload[PAYLOAD_SIZE];

void setup()
{

  Serial.begin(9600);
  Serial.println("SLAVE SENDER NODE");
  Serial.print("Node address: ");
  Serial.println(NODE_ADDRESS);
  Serial.print("Payload size: ");
  Serial.println(PAYLOAD_SIZE);
  Serial.println("**********************");

  Wire.begin(NODE_ADDRESS);  // Activate I2C network
  Wire.onRequest(requestEvent); // Request attention of master node
}

void loop()
{
  delay(100);
  nodePayload[0] = NODE_ADDRESS; // I am sending Node address back.  Replace with
  nodePayload[1] = analogRead(A0)/4; // Read A0 and fit into 1 byte. Replace this

}

void requestEvent()
{
  Wire.write(nodePayload,PAYLOAD_SIZE);
  Serial.print("Sensor value: ");  // for debugging purposes.
  Serial.println(nodePayload[1]); // for debugging purposes.
}
```

I2C Resources

- Arduino Wire (I2C) Library (http://arduino.cc/en/reference/wire)
- I2C on Wikipedia (http://en.wikipedia.org/wiki/I%C2%B2C)
- I2C Manual by NXP (http://www.nxp.com/campaigns/i2c-bus/)
